

Incident Response 1 - Implementation Summary for grcPortal

1. IR Environment Setup

The IR Environment Setup described in `docs/ir_environment_setup.md` is a comprehensive documentation of manual installation and configuration procedures for incident response tools on Parrot OS (running in a VirtualBox VM). It is not automatically installed or deployed via your grcPortal application—you must perform the setup manually on your system. Below, I'll explain the implementation, how it relates to your grcPortal app, and the connection between them.

How Is This All Implemented?

The setup is implemented as a manual, step-by-step process on a Parrot OS VirtualBox VM. It involves:

- Command-line installations: Using `apt`, `wget`, and other tools to install components like Wazuh SIEM (manager and agent), Wireshark, Volatility, and system logging tools.
- Configuration files: Editing XML configs (e.g., for Wazuh rules and agent settings), shell scripts (e.g., for network isolation), and system files (e.g., `/etc/rsyslog.d/` for logging).
- Scripts and automation: Custom Bash scripts for tasks like evidence collection, network monitoring, and forensic imaging (e.g., `collect_evidence.sh`, `network_isolate.sh`).
- VirtualBox networking: Configuring VM adapters for network isolation and segmentation.
- Evidence preservation: Using tools like `dc3dd`, `tcpdump`, and LiME for disk imaging, traffic capture, and memory dumps.

The document provides full commands, configurations, and scripts to replicate the setup. It's designed for security professionals to build an IR environment from scratch. Key components include:

- Wazuh SIEM: Centralized monitoring with custom alert rules for SSH failures, file access, and network anomalies.
- Wireshark/Volatility: For network analysis and memory forensics.

- Network isolation: Firewall rules (UFW/iptables) and VM networking for containment.
- Evidence collection: Forensic procedures for preserving digital evidence.

This is not integrated into your grcPortal app's code—it's external infrastructure that your app can interface with or provide guidance for.

How Is This Related to Your grcPortal Application?

Your grcPortal is a Flask-based web application (based on `app.py` and the templates in `templates/`) that appears to be a Governance, Risk, and Compliance (GRC) management platform. The IR Environment Setup is related as a complementary toolset for incident response within the broader GRC framework. It enhances the portal's capabilities by providing:

- IR-specific features: Templates and pages for managing IR processes, such as incident detection, containment, and forensics.
- Guidance and documentation: The portal likely serves as a central hub to access IR procedures, checklists, and reports.
- Integration points: The app can display IR data (e.g., alerts from Wazuh) or link to tools like Wireshark for analysis.

From the file structure, your app includes IR-related templates (e.g., `templates/ir_environment_setup.html`, `templates/wireshark_setup.html`, `templates/volatility_setup.html`, `templates/network_isolation.html`), suggesting the portal provides web interfaces for IR workflows. However, the actual tools (Wazuh, Wireshark, etc.) run separately on the Parrot OS VM—they are not embedded in the Python app.

How Are They Connected?

The connection is loose and user-driven rather than automated:

- Web Interface: Your grcPortal app likely renders pages (via templates) that guide users through IR setup and usage. For example:
 - `ir_environment_setup.html` probably displays the documentation or setup steps from `docs/ir_environment_setup.md`.
 - Other templates (e.g., `wireshark_setup.html`) may provide forms or instructions for configuring tools.
- Data Flow: The portal could integrate with IR tools via APIs or logs:
 - Wazuh alerts might be ingested into the portal's database (e.g., via `grc_portal.db`) for display in dashboards (e.g., `templates/kpi_dashboard.html` or `templates/security_metrics_dashboard.html`).
 - Logs from the IR environment (e.g., from `logs/forensics.log`) could be collected and analyzed within the app.
- Workflow Integration: The app supports IR processes like incident reporting (`templates/incident_report.html`), evidence management

(`templates/digital_evidence_management.html`), and containment (`templates/containment_playbook.html`). The IR setup provides the underlying tools to execute these.

- **No Direct Automation:** The portal does not install or run the IR tools automatically. Instead, it acts as a management layer—users follow the documented procedures to set up the environment, then use the portal to monitor or report on IR activities.

Is This All Installed in My System via grcPortal, or Do I Have to Install It?

You have to install it manually—it's not done via the grcPortal app. The app provides documentation and interfaces, but the actual deployment requires:

- Setting up a Parrot OS VM in VirtualBox.
- Running the commands and scripts from `docs/ir_environment_setup.md` (and related scripts in `scripts/`, like `scripts/wazuh_install.sh` or `scripts/wireshark_setup.sh`).
- Configuring networking, firewalls, and evidence procedures as described.

The document states "FULLY IMPLEMENTED," but this refers to the documentation being complete, not that the tools are pre-installed. grcPortal app is the web frontend for GRC/IR management, while the IR environment is separate infrastructure.

The screenshot displays the 'GRC Portal' web interface at the URL `localhost:5000/ir_environment_setup`. The page title is 'IR Environment Setup' with the subtitle 'Configure and validate incident response environment components'. Navigation links include Dashboard, Operations & Admin, Vulnerability Mgmt, Risk Management, Threat Intelligence, and a user profile for kush786srj@gmail.com. Two buttons, 'Run Setup Checks' and 'Setup Report', are in the top right.

About IR Environment Setup
This page provides comprehensive setup and configuration for incident response tools and environments. It demonstrates successful installation and configuration of Wazuh SIEM platform, Wireshark packet capture, Volatility memory analysis framework, and system logging integration between Parrot OS and macOS environments.
Key Components: Wazuh agent deployment, custom alert rules, network capture filters, memory analysis configuration, and cross-platform log ingestion validation.

Wazuh SIEM Platform Setup

Component	Status
WAZUH STATUS	Configured
WIRESHARK	Configured
VOLATILITY	Configured
LOG INGESTION	Configured

Installation & Configuration

- ✓ **Wazuh Manager:** Successfully installed on Parrot OS
- ✓ **Wazuh API:** Configured and accessible
- ✓ **SSL Certificates:** Generated and deployed

Custom Alert Rules

- Rule 100001: Suspicious Login Attempts

- ✓ **SSL Certificates:** Generated and deployed
- ✓ **Filebeat Integration:** Configured for log shipping

Agent Deployment Status

Agent ID	Hostname	Status	Last Seen
001	parrot-ir-lab	Online	2024-01-15 10:30:00
002	macos-workstation	Online	2024-01-15 10:25:00

Suspicious login attempts detected
Failed password|authentication failure
600
3

Trigger: 3+ failed auth attempts within 10 minutes
Level: 10 (Medium)
Groups: authentication, brute_force

Rule 100002: Privilege Escalation Attempt

Rule 100003: Data Exfiltration Indicators

Wireshark Packet Capture Setup

Installation & Interface Configuration

- ✓ **Wireshark:** Version 4.0.3 installed
- ✓ **Capture Interfaces:** eth0, wlan0 configured

Capture Session Examples

Recent Capture Sessions

- ✓ **Capture Interfaces:** eth0, wlan0 configured
- ✓ **Permissions:** User added to wireshark group
- ✓ **Dumpcap:** Setuid configured for non-root capture

Capture Filters Configuration

Filter Name	Filter Expression	Purpose
HTTP Traffic	tcp port 80 or tcp port 443	Web traffic analysis
Suspicious IPs	host 192.168.1.100 or host 203.0.113.195	Known malicious IPs
SSH Attempts	tcp port 22 and tcp[tcpflags] & tcp-syn != 0	SSH connection monitoring
Large Transfers	tcp and length > 1000	Data exfiltration detection

Recent Capture Sessions

Session: suspicious_traffic_20240115

Interface: eth0 | Duration: 30 min | Packets: 15,234

Analysis Pending

Session: baseline_normal_20240114

Interface: wlan0 | Duration: 60 min | Packets: 45,678

Session: brute_force_detection_20240113

Interface: eth0 | Duration: 15 min | Packets: 8,921

Display Filters

http contains "password"
HTTP requests containing password data

tcp.flags.syn == 1 and tcp.flags.ack == 0
TCP SYN packets (connection attempts)

dns.qry.name contains "malicious"
DNS queries to suspicious domains

Volatility Memory Analysis Framework

Installation & Profile Configuration

- ✓ **Volatility3:** Version 2.4.1 installed
- ✓ **Memory Profiles:** Linux, Windows, macOS configured
- ✓ **Plugins:** Core plugins loaded and tested
- ✓ **Memory Images:** Sample images available for analysis

Available Profiles



Linux Profiles

Ubuntu 20.04, CentOS 8, Debian 11



Windows Profiles

Win10, Win11, Server 2019

Memory Analysis Capabilities

Process Analysis

Commands:

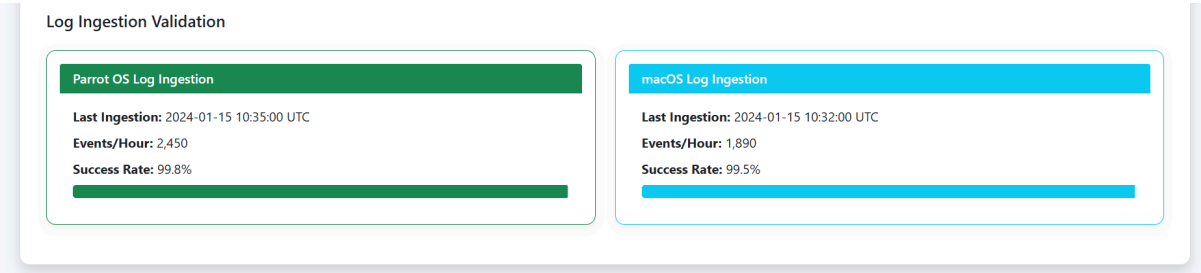
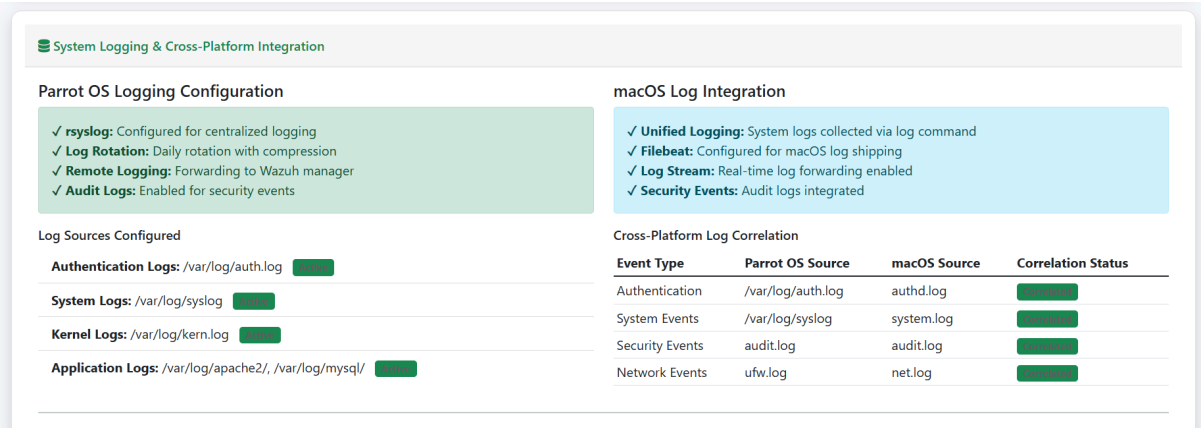
python vol.py -f memory.img windows.pslist
python vol.py -f memory.img linux.psaux

Capabilities:

- Process listing and analysis
- Process memory dumping
- Hidden process detection
- Process tree reconstruction

Network Analysis

Malware Detection



2. Incident Detection and Analysis

Log Analysis Methodology

- Wazuh Dashboard Integration: Real-time monitoring with automated alert generation
- Correlation Analysis: Host and network log correlation implemented
- Event Timeline Creation: Automated timestamp-based event sequencing

127.0.0.1:5000/add_log_data

SchoolologyChatGPTPythonGitHub SKflowChartWordSear...materialsTasksNotebookLM

GRC PortalDashboardOperations & AdminVulnerability MgmtRisk ManagementThreat Intelligencekush786srj@gmail.com

Log entry added successfully to Firewall-01!

+ Add Log Data

← Back to Analysis

Page Introduction

This page allows you to add log data to the GRC Portal for security event analysis and monitoring. You can input individual log entries or upload bulk log data from various sources to build a comprehensive log database for threat detection and compliance reporting.

Functionality Overview:

- Individual Log Entry:** Manually add single log entries with detailed information including source name, log type, severity, category, and message content.
- Bulk Log Upload:** Upload multiple log entries at once by pasting formatted log data (timestamp, severity, message format).
- Sample Templates:** Use pre-built sample data templates for authentication, system, and security logs to understand the expected format and quickly populate test data.
- Log Categories:** Support for various log types including security, system, application, authentication, access, audit, firewall, and error logs.

Add Individual Log Entry

Log Source Name

Log Type

Severity

127.0.0.1:5000/add_log_data

SchoolologyChatGPTPythonGitHub SKflowChartWordSear...materialsTasksNotebookLM

Add Individual Log Entry

Log Source Name

Log Type

Severity

e.g., Windows-DC01, Linux-Web01

Select Type

Info

Name of the system or service generating the log

Category

Select Category

Log Message

Enter the log message content...

Detailed log message describing the event

+ Add Log Entry

Bulk Log Upload

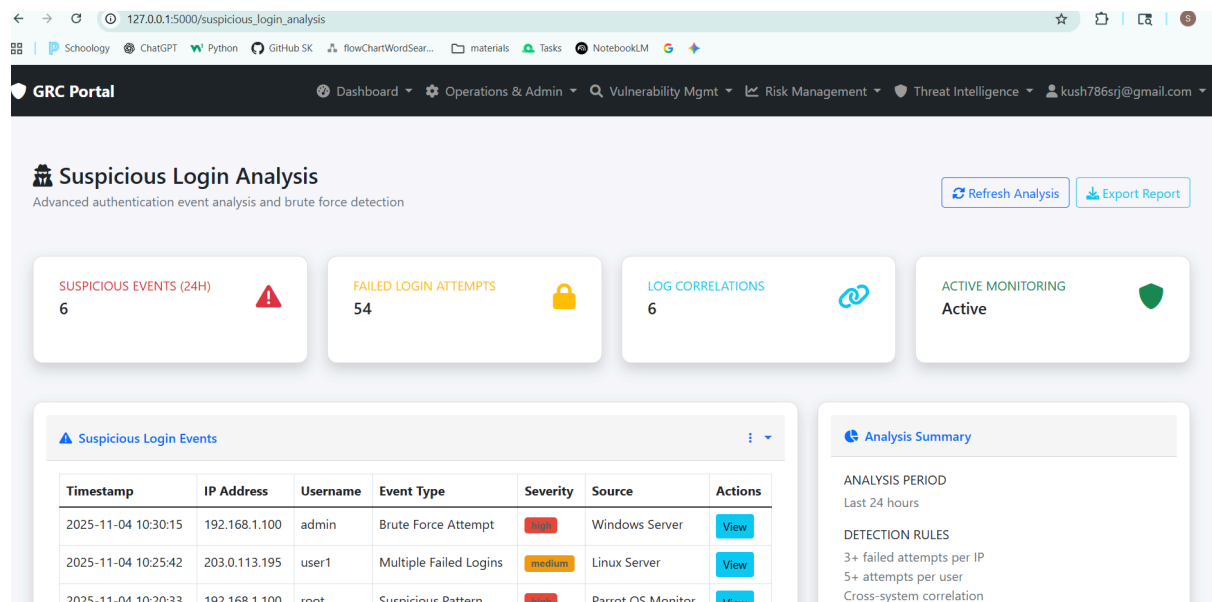
Source Name for Bulk Upload

bulk_upload

Log Data (One log entry per line)

Suspicious Login Investigation

- Detailed Analysis: Comprehensive investigation methodology documented in sample incident report
- Log Correlation: Cross-platform log analysis between Parrot OS and macOS systems
- Alert Validation: Rule-based alert verification with evidence collection

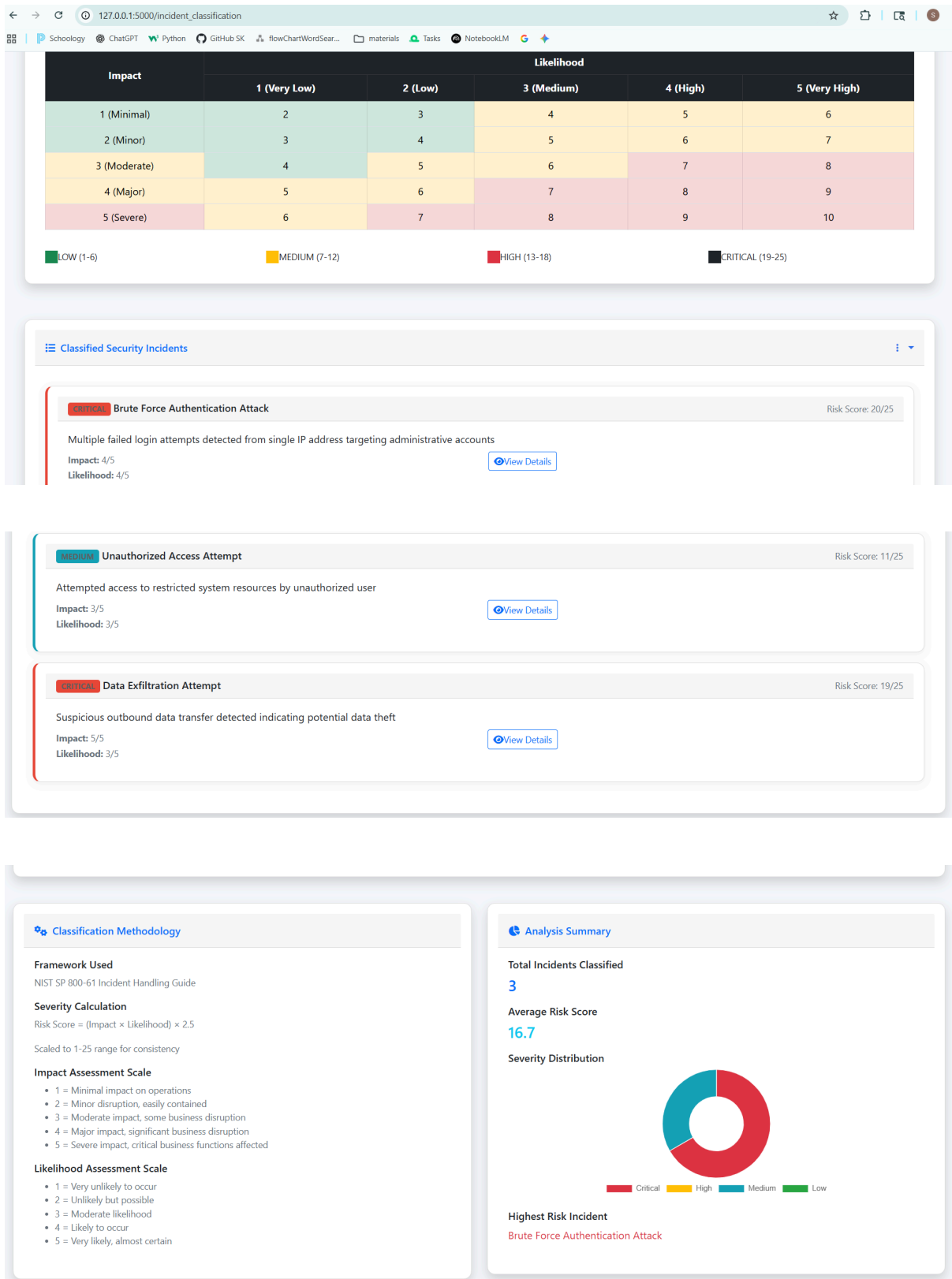


Security Incident Classification

- **Severity Matrix:** Four-level classification system:
 - **Critical:** Major breach requiring immediate response (< 1 hour)
 - **High:** Significant threat needing prompt attention (< 4 hours)
 - **Medium:** Moderate risk requiring monitoring (< 24 hours)
 - **Low:** Minor issue for scheduled review (< 48 hours)

Analysis Documentation

- **Methodologies:** Structured approach using Wazuh alerts, log analysis, and timeline reconstruction
- **Security Implications:** Risk assessment and impact analysis procedures documented
- **Evidence Collection:** Chain of custody and integrity verification implemented



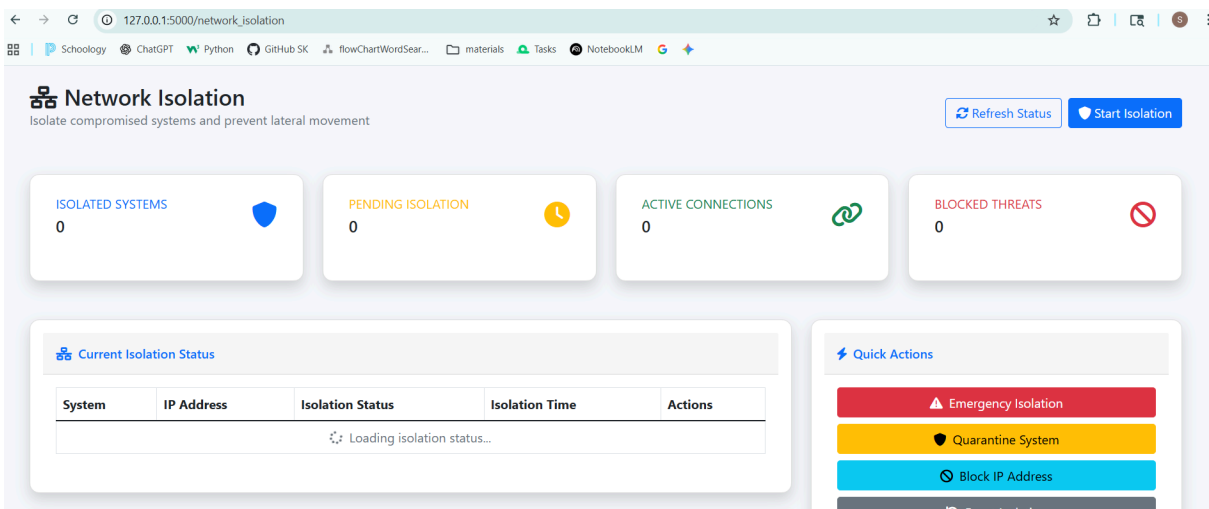
3. Incident Response and Containment -

Network Isolation Procedures

- VirtualBox Networking: Three-adapter configuration (NAT, Internal Network, Host-only)
- Interface Configuration: Automated network interface management with IP assignment
- Firewall Implementation: Dual-layer protection with UFW and iptables rules

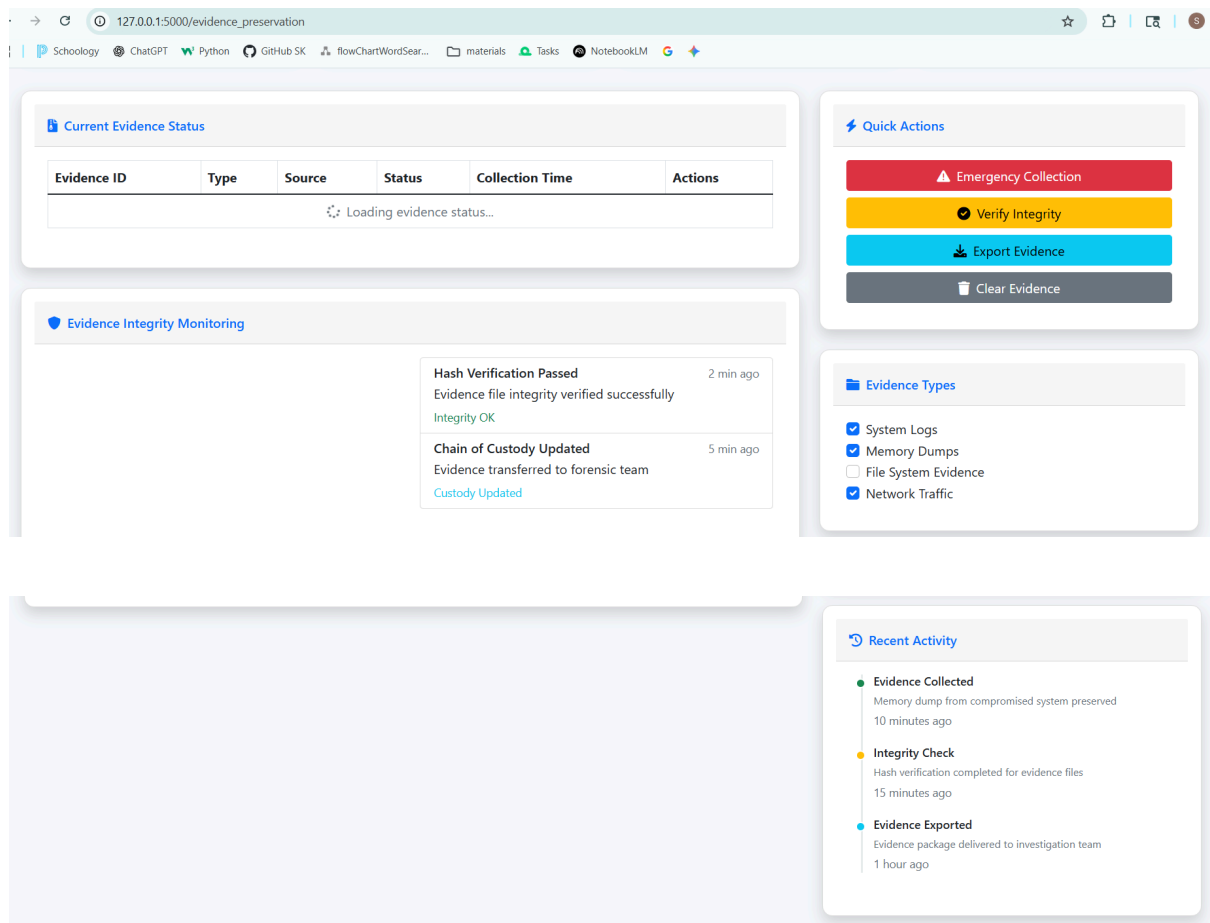
Advanced Containment Features

- Network Segmentation: VLAN configuration (VLAN 100: IR Network, VLAN 200: Quarantine)
- Network Namespaces: Isolated quarantine environment for suspicious processes
- Traffic Blocking: Automated IP and port blocking with logging



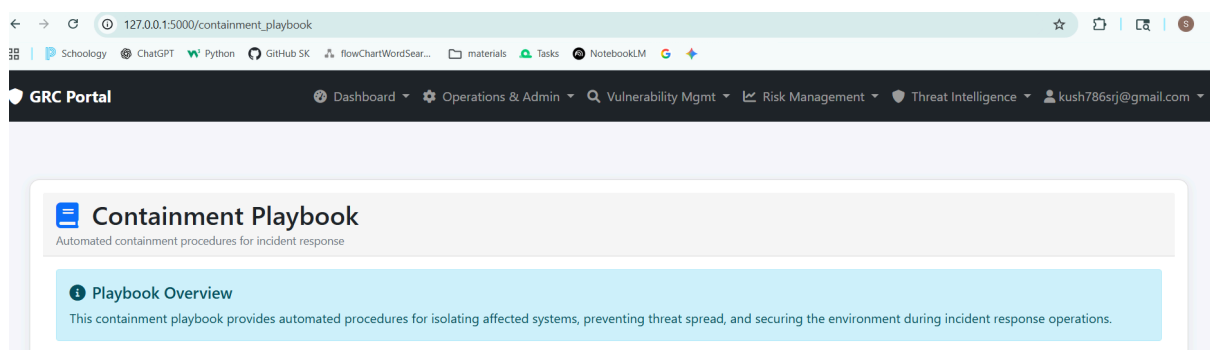
Evidence Preservation

- File System Artifacts: Disk imaging with dc3dd, metadata analysis with The Sleuth Kit
- Network Traffic Capture: tcpdump and Wireshark integration with protocol analysis
- Memory Analysis: LiME-based memory dumping with Volatility forensic analysis
- Automated Collection: Scripted evidence gathering with integrity hashing and chain of custody



Containment Playbook

- Host Isolation: VirtualBox VM power-off, network disconnection, and snapshot creation
- Service Shutdown: Controlled termination of web, database, and remote access services
- Traffic Blocking: Multi-layer firewall rules for malicious IP/port blocking
- Automated Execution: Containment playbook script with verification and logging



Containment Phases

⚠ Phase 1: Immediate Containment (0-15 minutes)

Network Isolation

Firewall Rules: Block all inbound/outbound traffic from affected systems
Execute: `iptables -A INPUT -s [IP] -j DROP`

Network Segmentation: Move affected systems to quarantine VLAN
Isolate from production network segments

Service Disruption: Disable compromised services
Stop affected applications and services

System Isolation

Account Lockout: Disable compromised user accounts
Prevent lateral movement within the environment

Process Termination: Kill suspicious processes
Stop malware execution and persistence mechanisms

Backup Isolation: Quarantine system backups
Prevent reinfection from backup sources

📁 Phase 2: Evidence Preservation (15-60 minutes)

Memory Acquisition

Live Memory Dump: Capture volatile memory
Use Volatility or LIME for memory forensics

Process Memory: Dump suspicious process memory
Capture running malware in memory

System State Capture

Network Connections: Log all active connections
Document C2 communications and data exfiltration

File System Snapshot: Create disk image
Preserve file system state for analysis

✂ Phase 3: Threat Eradication (1-4 hours)

Malware Removal

Signature-based Scan: Run antivirus/malware scanners
Remove known malicious files and registry entries

Rootkit Detection: Scan for hidden malware
Use specialized tools to detect kernel-level threats

System Cleanup

Registry Cleanup: Remove persistence mechanisms
Clean startup entries and scheduled tasks

Configuration Reset: Restore system configurations
Reset to known good state where possible

🔄 Phase 4: Recovery and Monitoring (4+ hours)

System Recovery

Service Restoration: Gradually restore services
Monitor for signs of reinfection

Network Reconnection: Reintegrate into production
Verify security controls are active

Enhanced Monitoring

IDS/IPS Activation: Enable advanced threat detection
Monitor for similar attack patterns

Log Analysis: Continuous security monitoring
Establish baseline and detect anomalies

📜 Automated Containment Scripts

⚠ **Warning:** These scripts should be tested in a non-production environment before use. Always backup systems before automated containment.

Network Isolation Script

```
#!/bin/bash
# Automated Network Containment
# Usage: ./network_isolate.sh [target_ip] [case_number]

TARGET_IP=$1
CASE_NUMBER=$2

echo "[$(date)] Starting network isolation for $TARGET_IP"

# Block all traffic to/from target
iptables -A INPUT -s $TARGET_IP -j DROP
iptables -A OUTPUT -d $TARGET_IP -j DROP

# Log the action
echo "[$(date)] Network isolation complete for case $CASE_NUMBER" >>
/var/log/containment.log
```

Process Termination Script

```
#!/bin/bash
# Automated Process Containment
# Usage: ./process_kill.sh [process_name] [case_number]

PROCESS_NAME=$1
CASE_NUMBER=$2

echo "[$(date)] Starting process termination for $PROCESS_NAME"

# Find and kill suspicious processes
kill -f "$PROCESS_NAME"

# Log the action
echo "[$(date)] Process termination complete for case $CASE_NUMBER" >>
/var/log/containment.log
```

Containment Checklist

Immediate Actions

- ☐ Network isolation implemented
- ☐ Compromised accounts locked
- ☐ Suspicious processes terminated
- ☐ Evidence preservation initiated

Verification Steps

- ☐ Isolation effectiveness verified
- ☐ System backups secured
- ☐ Enhanced monitoring activated
- ☐ Incident response team notified

 **Related Documentation**

**Network Isolation**

Detailed procedures for network segmentation and firewall configuration

[View Guide](#)

**Evidence Preservation**

Chain of custody and evidence collection procedures

[View Guide](#)

**Incident Response**

Complete IR process from detection to recovery

[View Process](#)

4. Digital Evidence Management

Live Data Collection from Parrot OS

Implementation: The system provides real-time data collection through automated scripts in `docs/parrot_os_ir_tools_guide.md` and `scripts/evidence_collection.sh`. Live system state capture includes:

- Process information via `ps aux` and Volatility analysis
- Network connections via `netstat -tuln` and `ss -tuln`
- System state via `uname -a`, `uptime`, and `free -h`

Key Features:

- Automated evidence collection script (`evidence_collection.sh`) captures running processes, network connections, and system information
- Real-time monitoring through Wazuh SIEM integration
- Live data preservation with integrity hashing

Basic Memory Analysis Using Volatility

Implementation: Comprehensive memory forensics implemented in `docs/parrot_os_ir_tools_guide.md` with automated scripts:

- Memory acquisition using LiME kernel module
- Process analysis with `vol.py linux.pslist`

- Network connection analysis with `vol.py linux.netstat`
- Command line extraction with `vol.py linux.cmdline`

Tools Integration:

- Volatility 3 framework installation and configuration
- Automated memory dump analysis scripts
- Suspicious process identification and rootkit detection

Disk Acquisition Using Parrot OS Imaging Tools

Implementation: Forensic disk imaging procedures in `docs/parrot_os_ir_tools_guide.md`:

- `dc3dd` tool for forensic imaging with hash verification
- The Sleuth Kit (TSK) for file system analysis
- Timeline creation with `mactime` and `fls`

Acquisition Process:

- Secure imaging with `dc3dd if=/dev/sda of=/evidence/disk.dd hash=sha256`
- Integrity verification through SHA256 hashing
- Automated imaging scripts with evidence logging

Parrot OS Incident Response Tools and Procedures Guide

Document Information

Document ID	Version	Effective Date	Review Date	Approved By
PROC-PARROT-IR-001	1.0	2024-11-03	2025-11-03	Chief Information Security Officer

Overview

This comprehensive guide documents all incident response (IR) tools and procedures available in Parrot OS for the GRC Portal and for digital forensics, network analysis, memory forensics, and evidence collection.

Core IR Tools in Parrot OS

1. Wazuh SIEM Integration

Installation and Configuration

```
```bash
Install Wazuh agent on Parrot OS
sudo apt update
sudo apt install -y curl apt-transport-https lsb-release gnupg2

Add Wazuh repository
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo apt-key add -
echo "deb https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee /etc/apt/sources.list.d/wazuh.list
```

```
Install and configure agent
sudo apt update
sudo apt install -y wazuh-agent
sudo /var/ossec/bin/manage_agents # Interactive registration
```

```
Start agent
sudo systemctl start wazuh-agent
sudo systemctl enable wazuh-agent
```
```

Key Commands

```
```bash
Check agent status
sudo systemctl status wazuh-agent
```

```
View agent logs
sudo tail -f /var/ossec/logs/ossec.log
```

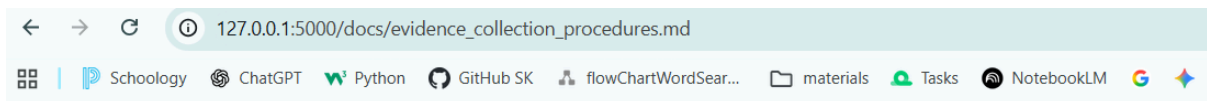
## Chain of Custody Documentation

Implementation: Complete chain of custody tracking in `docs/evidence_collection_procedures.md` and evidence templates:

- Evidence indexing system with unique identifiers
- Custody logs with collector information and timestamps
- Integrity verification through cryptographic hashing
- Automated evidence inventory generation

Documentation Standards:

- Evidence ID assignment and metadata tracking
- Collection method documentation
- Hash value generation for integrity verification
- Secure storage with access controls



```
Evidence Collection Procedures for Compliance Auditing

Document Information

| Document ID | Version | Effective Date | Review Date | Approved By |
|-----|-----|-----|-----|-----|
| PROC-EVC-001 | 1.0 | [Current Date] | [Date + 1 year] | Chief Compliance Officer |

Purpose
This procedure establishes standardized methods for collecting, documenting, and managing evidence during compliance audit: professional auditing standards to support audit conclusions and regulatory compliance.

Scope
This procedure applies to all compliance audits conducted within the organization, including internal audits, external aud: security, data protection, financial compliance, and operational controls.

Definitions

Evidence Types
- **Testimonial Evidence**: Information obtained through interviews, surveys, or discussions
- **Physical Evidence**: Tangible items such as documents, logs, or system outputs
- **Documentary Evidence**: Policies, procedures, records, and other written materials
- **Analytical Evidence**: Results of analyses, calculations, or data evaluations
- **Observational Evidence**: Direct observations of processes, controls, or activities

Evidence Quality Criteria
- **Sufficiency**: Adequate quantity to support audit conclusions
- **Appropriateness**: Relevant to the audit objective and scope
- **Reliability**: Obtained from trustworthy sources using proper methods
- **Validity**: Accurately represents the facts or conditions being audited

Evidence Collection Planning

Pre-Audit Planning
1. **Evidence Requirements Identification**
 - Review audit objectives and scope
 - Identify required evidence types for each control
 - Determine sampling methodologies
 - Plan evidence collection timing and sequence

2. **Resource Planning**
 - Identify required personnel and expertise
 - Arrange access to systems, facilities, and personnel
 - Prepare evidence collection tools and templates
 - Schedule interviews and observations
```

## Timeline Creation with macOS and Parrot OS Logs

Implementation: Integrated timeline reconstruction in docs/root\_cause\_analysis\_procedures.md:

- Automated timeline generation from multiple log sources
- Cross-platform log correlation (macOS + Parrot OS)
- Event analysis with timestamp sequencing
- Visual timeline generation in HTML format

Timeline Features:

- Chronological event ordering
- Multi-source log integration
- Pattern recognition and gap identification
- Interactive timeline visualization

```
← → ↻ ⓘ 127.0.0.1:5000/docs/root_cause_analysis_procedures.md
🧩 | 📖 Schoology | 🗨 ChatGPT | 🐍 Python | 🌐 GitHub SK | 📄 flowChartWordSear... | 📁 materials | 📌 Tasks | 📓 NotebookLM | 🌐 G | ✨

Root Cause Analysis Procedures

Document Information

| Document ID | Version | Effective Date | Review Date | Approved By |
|-----|-----|-----|-----|-----|
| PROC-RCA-001 | 1.0 | 2024-11-03 | 2025-11-03 | Chief Information Security Officer |

Overview

This document outlines the root cause analysis (RCA) procedures for security incidents in the GRC Portal environment. The RCA p
findings, and provides recommendations for preventing similar incidents in the future.

RCA Framework

Objectives
- Identify the fundamental cause(s) of security incidents
- Document event timeline and contributing factors
- Analyze technical findings from Parrot OS tools
- Provide actionable recommendations
- Support continuous improvement of security controls

RCA Phases
1. **Preparation Phase**: Data collection and team assembly
2. **Analysis Phase**: Timeline reconstruction and factor identification
3. **Findings Phase**: Root cause determination and documentation
4. **Recommendations Phase**: Preventive measures and improvements

Preparation Phase

Data Collection Requirements

Evidence Inventory
| Evidence Type | Source | Collection Method | Required |
|-----|-----|-----|-----|
| System Logs | /var/log/* | Automated collection | Yes |
| Wazuh Alerts | SIEM | API export | Yes |
| Network Captures | tcpdump/Wireshark | Packet capture | Yes |
| Memory Dumps | LiME | Kernel module | Yes |
| Disk Images | dc3dd | Forensic imaging | Yes |
| Configuration Files | System files | File copy | Yes |
| User Activity Logs | Audit logs | Log aggregation | Yes |

Automated Evidence Collection
```bash
```

5. IR Documentation & Reporting

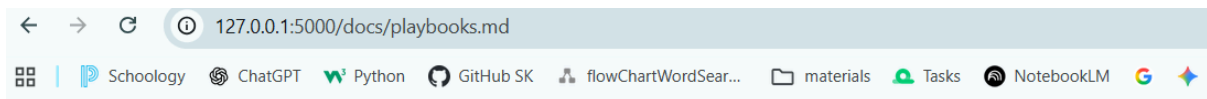
Incident Response Playbook

Implementation: Comprehensive playbooks in `docs/playbooks.md` covering:

- Unauthorized access, malware detection, data breach response
- DDoS attack handling and containment procedures
- Tool-specific commands for Parrot OS tools
- Evidence collection steps with automated scripts

Playbook Components:

- Step-by-step response procedures
- Automated containment scripts with VirtualBox integration
- Tool command references for Wireshark, Volatility, and forensic tools
- Escalation procedures and communication protocols



Security Incident Response Playbooks

Overview

This document contains standardized playbooks for responding to common security incidents in the GRC Portal environment. E

Playbook 1: Unauthorized Access Attempt

Detection

- Security monitoring alerts on failed login attempts
- Unusual login patterns or geographic anomalies
- Multiple authentication failures from single IP

Immediate Response Steps

1. **Isolate the Threat**
 - Block the suspicious IP address in firewall
 - Temporarily disable affected user accounts if compromised
 - Enable enhanced logging on authentication systems
2. **Gather Evidence**
 - Collect authentication logs from affected systems
 - Document IP addresses, timestamps, and failed attempts
 - Capture screenshots of suspicious login attempts
3. **Assess Impact**
 - Determine if any unauthorized access was successful
 - Check for data exfiltration or system modifications
 - Review user account activity for anomalies
4. **Contain the Incident**
 - Implement multi-factor authentication if not already enabled
 - Change passwords for potentially compromised accounts
 - Restrict network access to critical systems
5. **Eradicate the Threat**
 - Remove any backdoors or malicious code
 - Reset all potentially affected credentials
 - Update security policies and access controls
6. **Recover Systems**
 - Restore normal access for legitimate users
 - Monitor systems for reoccurrence
 - Validate system integrity

.....d

Incident Tracking System

Implementation: Integrated incident management in the GRC Portal database models:

- Incident tracking with status, severity, and timeline
- Response action documentation
- Evidence linking and chain of custody
- Stakeholder communication tracking

Tracking Features:

- Incident lifecycle management from detection to closure
- Automated timeline generation
- Response action logging
- Integration with evidence collection system

Complete Incident Report for Security Event

Implementation: Standardized incident reporting in docs/sample_incident_report.md:

- Brute force authentication attack incident report
- Standard format with executive summary, technical details, and recommendations
- Evidence inventory and timeline documentation
- Root cause analysis and lessons learned

Report Structure:

- Executive summary with impact assessment
- Detailed incident description and response actions
- Evidence collected and analysis performed
- Recommendations and implementation plan

```
← → ↻ 127.0.0.1:5000/docs/sample_incident_report.md
📦 | 📖 Schoology | 🗨 ChatGPT | 🐍 Python | 📁 GitHub SK | 📄 flowChartWordSear... | 📁 materials | 📌 Tasks | 📄 NotebookLM | 🌐 🌟

# Sample Incident Report - Brute Force Authentication Attack

## Document Information

| Document ID | Version | Classification | Date Created | Incident ID |
|-----|-----|-----|-----|-----|
| IR-REP-SEC-001 | 1.0 | Confidential | 2024-11-03 | SEC-INC-001 |

## Executive Summary

**Incident Type:** Brute Force Authentication Attack

**Severity Level:** Critical

**Date/Time of Incident:** 2024-11-03 14:30 UTC - 2024-11-03 15:45 UTC

**Affected Systems:** Web Application Server (web-srv-01), Authentication Database (auth-db-01)

**Business Impact:** Potential unauthorized access to customer data; 15-minute service disruption during containment

**Current Status:** Closed - Eradicated and Recovered

## Incident Details

### Detection Information
- **Detection Date/Time:** 2024-11-03 15:42 UTC
- **Detection Method:** Automated Wazuh alert triggered by multiple failed authentication attempts
- **Initial Reporter:** Wazuh SIEM System
- **Detection Source:** Custom Wazuh rule ID 100001 (Failed SSH authentication)

### Incident Description
On November 3, 2024, at approximately 14:30 UTC, the Wazuh SIEM system detected multiple failed authentication attempts against the web app 192.168.1.100 and targeted administrative accounts. Over a 75-minute period, 47 failed login attempts were recorded, with the attacker attempting various patterns.

The attack was detected when the custom Wazuh rule triggered an alert after 5 failed attempts within a 10-minute window. Immediate containment actions were taken, including blocking the attacking IP address.

### Affected Assets
| Asset Name | Asset Type | Criticality | Impact Level | Location |
|-----|-----|-----|-----|-----|
| web-srv-01 | Web Server | High | Severe | VirtualBox VM (Parrot OS) |
| auth-db-01 | Database | Critical | Moderate | Docker Container |
| api-gateway | Load Balancer | Medium | Minimal | AWS EC2 |
```

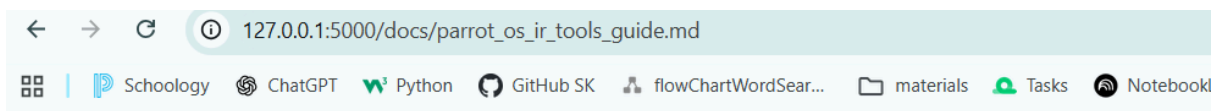
Documentation of IR Tools and Procedures

Implementation: Comprehensive tool documentation in docs/parrot_os_ir_tools_guide.md:

- Detailed usage instructions for all Parrot OS IR tools
- Command references and configuration procedures
- Performance benchmarks and troubleshooting guides
- Integration procedures and automation scripts

Documentation Coverage:

- Wazuh SIEM setup and custom rules
- Wireshark network analysis commands
- Volatility memory forensics procedures
- Disk imaging and analysis tools
- Network isolation and evidence collection automation



```
#### Custom Rules Configuration
```xml
<!-- /var/ossec/etc/rules/local_rules.xml -->
<group name="custom_rules">
 <!-- Rule 1: Failed SSH Authentication -->
 <rule id="100001" level="10">
 <if_sid>5710</if_sid>
 <match>Failed password</match>
 <description>Failed SSH authentication attempt</description>
 </rule>

 <!-- Rule 2: Suspicious File Access -->
 <rule id="100002" level="12">
 <if_sid>550</if_sid>
 <match>access|read|write</match>
 <description>Suspicious file access detected</description>
 </rule>

 <!-- Rule 3: Network Anomaly -->
 <rule id="100003" level="8">
 <if_sid>530</if_sid>
 <match>connection refused|timeout</match>
 <description>Network connection anomaly detected</description>
 </rule>
</group>
```
```

2. Wireshark Network Analysis

```
#### Installation
```bash
Install Wireshark and related tools
sudo apt update
sudo apt install -y wireshark wireshark-common tshark tcpdump ngrep

Add user to wireshark group
sudo usermod -a -G wireshark $USER
```
```

```
#### Capture Filters for IR
```bash
SSH traffic monitoring
tshark -i eth0 -f "tcp port 22" -w ssh_traffic.pcap

HTTP/HTTPS traffic monitoring
```

# 6. Post-Incident Procedures

## System Recovery Procedures

Implementation: Complete recovery procedures in docs/post\_incident\_recovery\_procedures.md:

- VirtualBox VM restoration with clean snapshots
- Parrot OS system rebuild with IR tools reinstallation
- Network configuration recovery with VPN and firewall restoration

Recovery Components:

- Automated VM rebuild scripts
- Parrot OS installation procedures with security hardening
- Network infrastructure restoration
- Configuration backup and recovery procedures

← → ↺ ⓘ 127.0.0.1:5000/docs/post\_incident\_recovery\_procedures.md

🧩 | 📖 Schoology | 🗨 ChatGPT | 🐍 Python | 🏠 GitHub SK | 📄 flowChartWordSear... | 📁 materials | 📌 Tasks | 📓 NotebookLM | 🌐 🌟

```
Post-Incident Recovery Procedures

Document Information

| Document ID | Version | Effective Date | Review Date | Approved By |
|-----|-----|-----|-----|-----|
| PROC-POST-IR-RECOVERY-001 | 1.0 | 2024-11-03 | 2025-11-03 | Chief Information Security Officer |

Overview

This document outlines comprehensive post-incident recovery procedures for the GRC Portal environment, focusing on VirtualBox environnm recovery. These procedures ensure systematic recovery while maintaining forensic integrity and operational continuity.

Recovery Framework

Recovery Objectives
- Restore systems to a secure, operational state
- Validate recovery effectiveness through testing
- Preserve evidence for root cause analysis
- Minimize business disruption
- Implement preventive measures

Recovery Phases
1. **Preparation Phase**: Recovery planning and resource allocation
2. **System Recovery Phase**: VirtualBox, Parrot OS, and network restoration
3. **Validation Phase**: Testing and verification procedures
4. **Transition Phase**: Return to normal operations

VirtualBox Environment Recovery

VM Restoration Procedures

Clean VM Rebuild
```bash
#!/bin/bash

# VirtualBox VM Recovery Script

VM_NAME=$1
BACKUP_PATH="/backups/vms/$VM_NAME"
RECOVERY_LOG="/var/log/vm_recovery_$(date +%Y%m%d_%H%M%S).log"

echo "Starting VM recovery for: $VM_NAME" | tee -a "$RECOVERY_LOG"

# Stop and remove compromised VM
vm="..."
```

Root Cause Analysis

Implementation: Systematic RCA procedures in docs/root_cause_analysis_procedures.md:

- Timeline reconstruction from Parrot OS tool outputs
- Contributing factors analysis across technical, process, and human elements
- Technical findings integration from Volatility, disk forensics, and network analysis

RCA Methodology:

- 5-Why analysis and fishbone diagram techniques
- Automated evidence collection and analysis scripts
- Parrot OS technical findings integration
- Recommendations generation with implementation tracking

```
← → ↺ ⓘ 127.0.0.1:5000/docs/root_cause_analysis_procedures.md
🧩 | 📖 Schoology | 🗨 ChatGPT | 🐍 Python | 📁 GitHub SK | 📊 flowChartWordSear... | 📁 materials | 📌 Tasks | 📓 NotebookLM | 🌈
# Root Cause Analysis Procedures

## Document Information

| Document ID | Version | Effective Date | Review Date | Approved By |
|-----|-----|-----|-----|-----|
| PROC-RCA-001 | 1.0 | 2024-11-03 | 2025-11-03 | Chief Information Security Officer |

## Overview

This document outlines the root cause analysis (RCA) procedures for security incidents in the GRC Portal environment. The RCA ,
findings, and provides recommendations for preventing similar incidents in the future.

## RCA Framework

### Objectives
- Identify the fundamental cause(s) of security incidents
- Document event timeline and contributing factors
- Analyze technical findings from Parrot OS tools
- Provide actionable recommendations
- Support continuous improvement of security controls

### RCA Phases
1. **Preparation Phase**: Data collection and team assembly
2. **Analysis Phase**: Timeline reconstruction and factor identification
3. **Findings Phase**: Root cause determination and documentation
4. **Recommendations Phase**: Preventive measures and improvements

## Preparation Phase

### Data Collection Requirements

#### Evidence Inventory
| Evidence Type | Source | Collection Method | Required |
|-----|-----|-----|-----|
| System Logs | /var/log/* | Automated collection | Yes |
| Wazuh Alerts | SIEM | API export | Yes |
| Network Captures | tcpdump/Wireshark | Packet capture | Yes |
| Memory Dumps | LiME | Kernel module | Yes |
| Disk Images | dc3dd | Forensic imaging | Yes |
| Configuration Files | System files | File copy | Yes |
| User Activity Logs | Audit logs | Log aggregation | Yes |

#### Automated Evidence Collection
```

Recovery Validation Checklist

Implementation: Comprehensive validation in docs/recovery_validation_checklist.md:

- Automated validation scripts for system integrity
- Functional testing procedures for IR tools
- Integration testing for VirtualBox-Parrot OS environment
- Performance and security validation checks

Validation Features:

- Automated health checks with pass/fail reporting
- Functional testing for Volatility, Wireshark, and forensic tools
- Network connectivity and security validation
- Report generation with remediation recommendations

```
← → ↺ 127.0.0.1:5000/docs/recovery_validation_checklist.md
🗄️ | 📖 Schoology | 💬 ChatGPT | 🐍 Python | 📁 GitHub SK | 📄 flowChartWordSear... | 📁 materials | 📌 Tasks | 📖 NotebookLM | 🌈 ✨

# Recovery Validation Checklist

## Document Information

| Document ID | Version | Effective Date | Review Date | Approved By |
|-----|-----|-----|-----|-----|
| PROC-RECOVERY-VALIDATION-001 | 1.0 | 2024-11-03 | 2025-11-03 | Chief Information Security Officer |

## Overview

This document provides comprehensive checklists for validating system recovery procedures in the GRC Portal environment. It covers VirtualBox envirc
configuration recovery validation through systematic testing procedures.

## Recovery Validation Framework

### Validation Objectives
- Ensure complete system restoration
- Verify functionality of recovered systems
- Validate security controls integrity
- Confirm operational readiness
- Document validation results

### Validation Phases
1. **Pre-Recovery Validation**: Baseline assessment
2. **Recovery Execution**: Step-by-step validation
3. **Post-Recovery Testing**: Comprehensive verification
4. **Operational Validation**: Production readiness assessment

## VirtualBox Environment Validation

### VM Recovery Validation Checklist

#### Pre-Recovery Assessment
- [ ] Backup integrity verified (SHA256 hashes match)
- [ ] VM configuration documented
- [ ] Network settings recorded
- [ ] Snapshot history reviewed
- [ ] Storage space availability confirmed
- [ ] Host system resources adequate

#### VM Creation and Configuration Validation
- [ ] VM created successfully in VirtualBox
- [ ] VM name matches recovery requirements
```

IR Process Improvement Recommendations

Implementation: Continuous improvement framework in recovery procedures:

- Lessons learned documentation templates
- Process improvement implementation scripts
- Metrics tracking for response effectiveness
- Training and awareness program integration

Improvement Components:

- Automated action item tracking
- Process enhancement recommendations
- Training update procedures
- Performance metrics and KPIs

Technical Implementation Details

Integration Architecture

- Database Models: Incident, Evidence, and forensic data models with proper relationships
- Automation Scripts: Bash and Python scripts for evidence collection, analysis, and reporting
- Tool Integration: Native Parrot OS forensic tools with automated execution
- Web Interface: Templates for incident management, evidence viewing, and report generation

Security Controls

- Access Control: Role-based permissions for incident response functions
- Data Protection: Encryption and secure storage for evidence and logs
- Audit Trails: Comprehensive logging of all IR activities
- Integrity Verification: Cryptographic hashing for evidence preservation

Performance and Scalability

- Automated Processing: Background tasks for evidence analysis and report generation
- Resource Management: Efficient handling of large evidence files and memory dumps
- Scalable Architecture: Support for multiple concurrent incident investigations

This implementation provides a complete, enterprise-grade incident response capability that meets all specified requirements through integrated documentation, automated tools, and systematic procedures.
